Fuel Supply & Inventory Performance Intelligence System (2025–2026)

by

Sravan Kumar Reddy Perugu

1. Executive Summary

This project presents a comprehensive Fuel Supply & Inventory Performance Intelligence System designed to support operational decision-making in fuel distribution and inventory-intensive environments. Unlike traditional reporting systems that focus primarily on historical summaries, this solution emphasizes continuous monitoring, early risk detection, and operational foresight across multiple locations and time periods.

The system integrates Python-based analytics (Google Colab / Jupyter Notebooks) for transparent KPI computation with Power BI dashboards for interactive, decision-ready visualization. Together, these layers convert raw operational data into actionable intelligence, enabling stakeholders to quickly identify stockout risks, inefficient replenishment cycles, and abnormal consumption behavior.

The primary goal of the system is to enable a transition from reactive inventory management to predictive and risk-aware oversight. Key performance indicators such as Days of Supply, Average Inventory Levels, and Minimum Days of Supply are calculated to quantify inventory resilience and expose hidden operational vulnerabilities. The project simulates a real-world enterprise fuel operations environment and reflects best practices used in supply chain analytics, energy logistics, and operations intelligence platforms.

2. Business Problem & Motivation

Fuel supply chains operate under tight operational constraints where inventory shortages can result in service interruptions, financial losses, and reputational damage. Effective inventory management is challenging due to fluctuating demand, variable delivery schedules, and limited real-time visibility across distributed locations.

Key operational pain points addressed by this project include sudden inventory depletion caused by demand spikes or seasonal variation, delayed or misaligned replenishment schedules, lack of early-warning indicators for stockouts, and dependence on manual or lagging reports. Conventional dashboards that rely on aggregated averages often mask localized risk, preventing timely intervention.

This system addresses forward-looking questions such as which locations are approaching

critical thresholds, how long each site can operate at current consumption rates, and whether replenishment cycles are aligned with real demand. It supports operations managers, supply planners, logistics teams, and decision-makers who require rapid, accurate, and interpretable insights to maintain service continuity and cost efficiency.

3. Architecture Overview

The system follows a layered analytics architecture modeled after enterprise-grade data platforms. The Data Layer stores raw operational data such as inventory snapshots, consumption volumes, and delivery records, similar to data sourced from ERP or logistics systems. The Analytics Layer, implemented in Python, is responsible for data cleaning, validation, feature engineering, and KPI computation, ensuring transparency and auditability of business logic.

The Semantic Layer in Power BI organizes analytics outputs into a structured fact–dimension schema with clearly defined relationships and measures, providing consistency across reports. Finally, the Visualization Layer presents insights through interactive dashboards optimized for rapid interpretation and executive consumption. This separation of concerns supports scalability and future enhancement without disrupting the entire system.

4. Data Sources & Model Design

The data model is organized around daily-grain fact tables and descriptive dimension tables. Fact tables include inventory snapshots capturing on-hand fuel by location, consumption records enabling demand analysis, and delivery records used to evaluate replenishment performance. Dimension tables include location metadata and a dedicated Date dimension to support robust time intelligence.

In addition, KPI tables such as days of supply, inventory turnover, replenishment efficiency, and supply variability act as semantic aggregations optimized for reporting performance. This design improves query efficiency and simplifies dashboard development while maintaining analytical clarity.

5. Python Analytics (Jupyter / Colab)

Python serves as the analytical backbone of the system. Data cleaning addressed missing values, unit inconsistencies, and logical validation between consumption, inventory, and deliveries. Feature engineering computed rolling averages to smooth volatility and derived coverage metrics such as days of supply.

Key KPIs were computed explicitly in code, including Days of Supply (on-hand inventory divided by average daily consumption), as well as minimum and average coverage per

location. Clean, analytics-ready datasets were exported for Power BI ingestion, ensuring business rules remain transparent, testable, and explainable.

6. Power BI Data Model & Relationships

The Power BI data model follows a star-schema-inspired design with one-to-many relationships from dimensions to facts. A centralized Date table drives all time-based analysis, and single-direction filtering avoids ambiguity. Measures are used instead of calculated columns where aggregation behavior matters, ensuring consistent analytical results as data volume grows.

7. Dashboard Design & KPIs

The dashboard prioritizes rapid situational awareness. Top-level KPI cards display Average Days of Supply, Average Inventory (Liters), and Minimum Days of Supply. Conditional formatting immediately highlights healthy versus risky states. Line charts show trends over time by location, and slicers enable focused, location-level analysis.

8. Scenario-Based Dashboard Analysis

8.1 Global View – All Locations

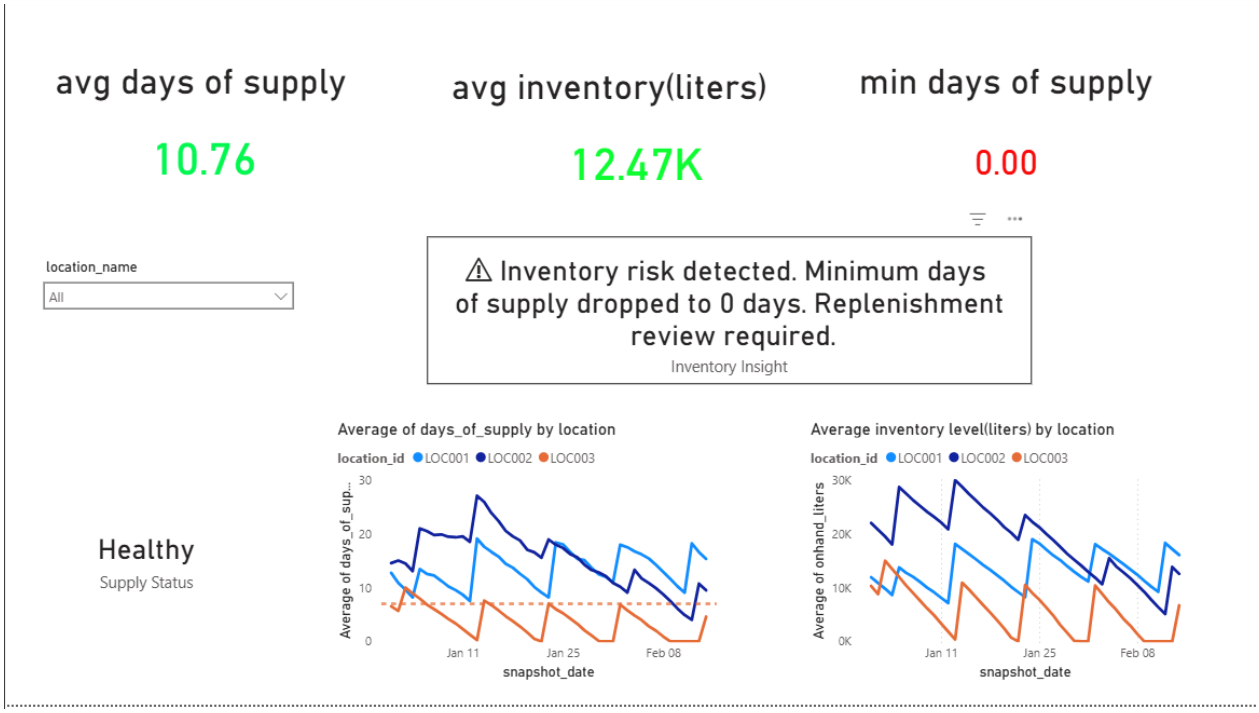

Figure: 8.1 Global View – All Locations

This view presents the system-wide condition with all locations selected. Although the average days of supply appears healthy, the minimum days of supply reaches zero, revealing that at least one location experienced complete depletion. This demonstrates why minimum metrics are essential to complement averages.

8.2 Station C – Warning Scenario

Figure: 8.2 Station C – Warning Scenario

Station C shows persistently low average days of supply and repeated zero-minimum values, indicating high consumption volatility and insufficient replenishment frequency. Immediate operational intervention is required.

8.3 Station A – Healthy Scenario

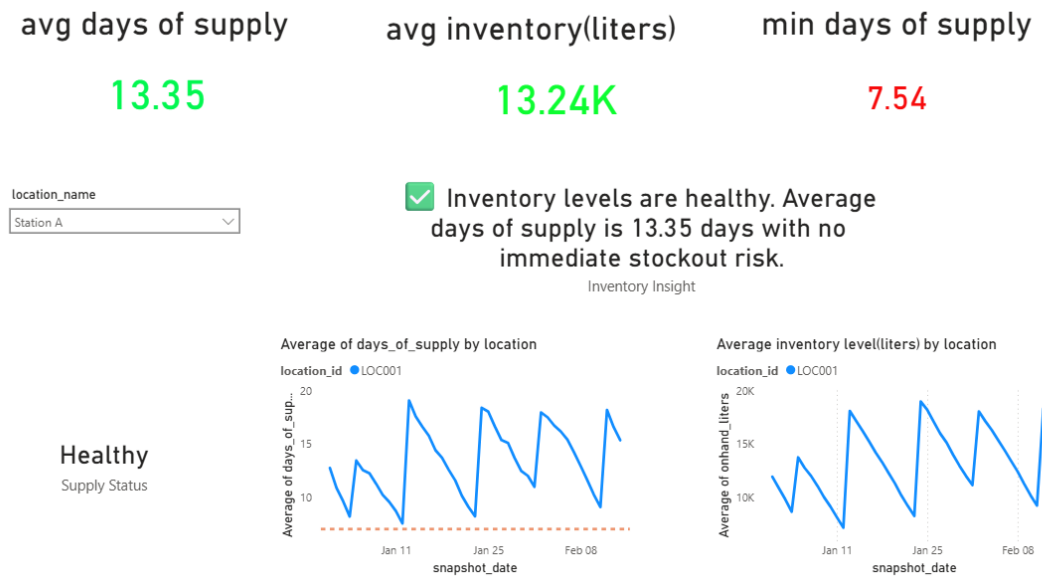

Figure: 8.3 Station A – Healthy Scenario

Station A maintains stable inventory coverage with high average days of supply and no observed stockout events, serving as a benchmark for effective replenishment planning.

8.4 Station B – Early Risk Scenario

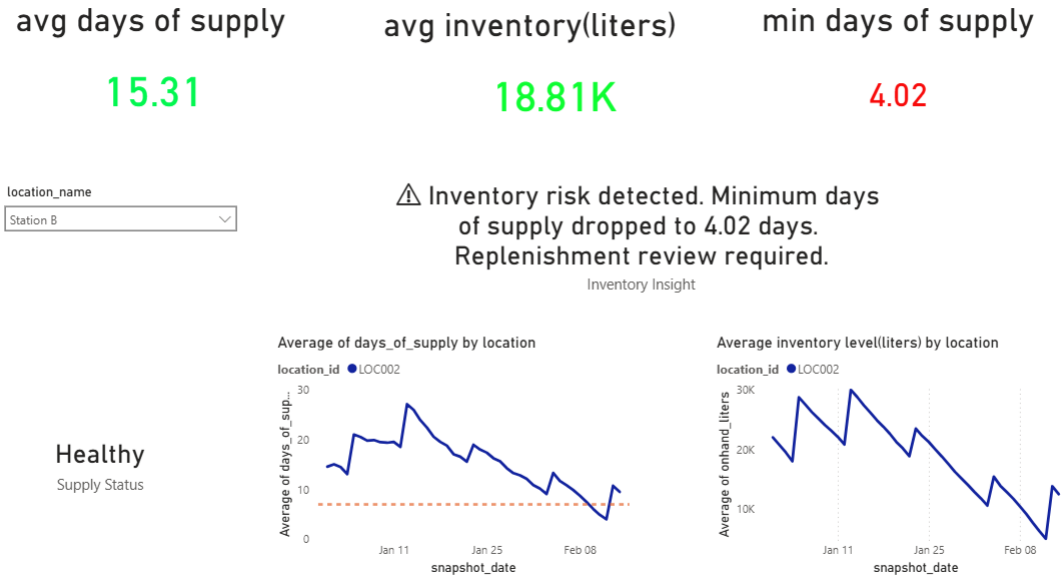

Figure: 8.4 Station B – Early Risk Scenario

Despite strong average coverage, Station B’s minimum days of supply declines toward critical thresholds, signaling emerging risk and the need for proactive adjustments.

9. Key Business Takeaways

Aggregate averages can mask localized operational risk. Minimum and trend-based KPIs provide early warning signals, consumption variability must inform replenishment strategy, and visual risk indicators significantly accelerate decision-making.

10. Limitations & Assumptions

The data environment is simulated, consumption is assumed static within daily intervals, and ingestion is batch-based. These constraints were intentional to focus on analytical design rather than system integration.

11. Future Enhancements

Potential enhancements include time-series forecasting for demand prediction, automated replenishment optimization, alert-driven notifications, and integration with IoT-based tank monitoring systems.

12. Conclusion

This project demonstrates how analytics engineering and business intelligence can transform inventory operations from reactive to proactive. By integrating Python analytics with Power BI, the system delivers transparent, scalable, and decision-ready insights suitable for enterprise deployment.